Assignment 2

CSA05 – Database Management Systems

Question No : 9

Name / Reg No: R DINESH (192524130)

1. DDL Commands – Create STUDENT, FACULTY and DEPARTMENT

Aim

To create the STUDENT, FACULTY and DEPARTMENT tables using DDL (CREATE TABLE) with the specified column names and data types.

Schema Description

Data types are written in MySQL. NUMBER(n) in the specification is an integer, so INT is used; a 10-digit mobile number does not fit in INT (maximum 2,147,483,647), so BIGINT is used. In Oracle use NUMBER(n) and VARCHAR2 instead.

Table	Column	Specified type	Type used (MySQL)
STUDENT	RegNo (Registration Number)	NUMBER(3)	INT
	Name (Student Name)	VARCHAR(15)	VARCHAR(15)
	Gender	CHAR(1)	CHAR(1)
	DOB (Date of Birth)	DATE	DATE
	MobileNo	NUMBER(10)	BIGINT
	City (Location of stay)	VARCHAR(15)	VARCHAR(15)
FACULTY	FacNo (Faculty Identifier)	VARCHAR(4)	VARCHAR(4)
	FacName	VARCHAR(15)	VARCHAR(15)
	Gender	CHAR(1)	CHAR(1)
	DOB	DATE	DATE
	DOJ (Date of Join)	DATE	DATE
	MobileNo	NUMBER(10)	BIGINT
DEPARTMENT	DeptNo (Department Identifier)	VARCHAR(4)	VARCHAR(4)
	DeptName	VARCHAR(15)	VARCHAR(15)
	DeptHead	VARCHAR(4)	VARCHAR(4)

Procedure / SQL Commands

```
CREATE DATABASE college;
USE college;

-- A. STUDENT table
CREATE TABLE STUDENT (
    RegNo     INT,
    Name      VARCHAR(15),
    Gender    CHAR(1),
    DOB       DATE,
    MobileNo  BIGINT,
```

```
    City      VARCHAR(15)
);

-- B. FACULTY table
CREATE TABLE FACULTY (
    FacNo     VARCHAR(4),
    FacName   VARCHAR(15),
    Gender    CHAR(1),
    DOB       DATE,
    DOJ       DATE,
    MobileNo  BIGINT
);

-- C. DEPARTMENT table
CREATE TABLE DEPARTMENT (
    DeptNo    VARCHAR(4),
    DeptName  VARCHAR(15),
    DeptHead  VARCHAR(4)
);

-- Verify the structures
DESC STUDENT;
DESC FACULTY;
DESC DEPARTMENT;
```

Output (DESC STUDENT):

Field	Type	Null	Key	Default	Extra
RegNo	int	YES		NULL	
Name	varchar(15)	YES		NULL	
Gender	char(1)	YES		NULL	
DOB	date	YES		NULL	
MobileNo	bigint	YES		NULL	
City	varchar(15)	YES		NULL	

Output (DESC FACULTY):

Field	Type	Null	Key	Default	Extra
FacNo	varchar(4)	YES		NULL	
FacName	varchar(15)	YES		NULL	
Gender	char(1)	YES		NULL	
DOB	date	YES		NULL	
DOJ	date	YES		NULL	
MobileNo	bigint	YES		NULL	

Output (DESC DEPARTMENT):

Field	Type	Null	Key	Default	Extra
DeptNo	varchar(4)	YES		NULL	
DeptName	varchar(15)	YES		NULL	
DeptHead	varchar(4)	YES		NULL	

Result: The STUDENT, FACULTY and DEPARTMENT tables were created successfully with the required structure. Constraints (primary and foreign keys) are added later in Question 3.

2. SQL Queries with Joins (EquiJoin, InnerJoin, OuterJoin)

Aim

To retrieve related data from several tables using equi join, inner join and outer join.

Supporting Setup (Assumption)

The three tables above cannot answer these queries on their own, so the following additions are assumed:

- FACULTY gets a DeptNo column (the department in which the faculty member works).
- COURSE (CourseNo, CourseName, FacNo) stores the course taken (handled) by each faculty member.
- ENROLLMENT (RegNo, CourseNo, Score) stores the courses a student has registered for; Score is NULL when no score has been recorded.

```
ALTER TABLE FACULTY ADD DeptNo VARCHAR(4);

CREATE TABLE COURSE (
    CourseNo    VARCHAR(4) PRIMARY KEY,
    CourseName  VARCHAR(30),
    FacNo       VARCHAR(4)
);

CREATE TABLE ENROLLMENT (
    RegNo     INT,
    CourseNo  VARCHAR(4),
    Score     INT NULL,
    PRIMARY KEY (RegNo, CourseNo)
);

INSERT INTO DEPARTMENT VALUES
('D001','CSE','F001'), ('D002','ECE','F002'),
('D003','MECH','F004'), ('D004','CIVIL','TBA');

INSERT INTO FACULTY VALUES
('F001','Ramesh','M','1975-04-12','2005-06-15',9876543210,'D001'),
('F002','Kamal','M','1978-09-23','2008-07-01',9876501234,'D002'),
('F003','Priya','F','1982-01-30','2012-08-20',9845012345,'D001'),
('F004','Meena','F','1980-11-05','2010-06-10',9811122233,'D003'),
('F005','Kamal','M','1985-02-17','2015-01-12',9822233344,'D001');

INSERT INTO STUDENT VALUES
(101,'Arun','M','2004-03-14',9000000001,'Chennai'),
(102,'Bhavana','F','2003-11-02',9000000002,'Madurai'),
(103,'Charan','M','2004-07-21',9000000003,'Trichy'),
(104,'Divya','F','2004-01-09',9000000004,'Coimbatore'),
(105,'Elango','M','2003-12-18',9000000005,'Salem'),
(106,'Gowri','F','2004-05-27',9000000006,'Erode');

INSERT INTO COURSE VALUES
('C001','DBMS','F001'), ('C002','Digital Electronics','F002'),
('C003','Signals and Systems','F002'), ('C004','Data Structures','F005'),
('C005','Thermodynamics','F004');

INSERT INTO ENROLLMENT VALUES
```

```
(101,'C001',85), (101,'C002',78), (102,'C001',92), (102,'C003',88),
(103,'C002',NULL), (104,'C004',67), (104,'C005',NULL), (105,'C003',NULL);
```

A. List the departments where the faculty members are working

(i) Equi join (join condition in WHERE with =)

```
SELECT DISTINCT d.DeptNo, d.DeptName
FROM   DEPARTMENT d, FACULTY f
WHERE  d.DeptNo = f.DeptNo;
```

(ii) Inner join

```
SELECT DISTINCT d.DeptNo, d.DeptName
FROM   DEPARTMENT d
INNER JOIN FACULTY f ON d.DeptNo = f.DeptNo;
```

Output:

DeptNo	DeptName
D001	CSE
D002	ECE
D003	MECH

(iii) Outer join (also shows departments that have no faculty)

```
SELECT d.DeptNo, d.DeptName, COUNT(f.FacNo) AS Faculty_Count
FROM   DEPARTMENT d
LEFT OUTER JOIN FACULTY f ON d.DeptNo = f.DeptNo
GROUP BY d.DeptNo, d.DeptName;
```

Output:

DeptNo	DeptName	Faculty_Count
D001	CSE	3
D002	ECE	1
D003	MECH	1
D004	CIVIL	0

Result: Equi join and inner join return only the departments D001, D002 and D003, which have faculty. The outer join also lists CIVIL (D004) with a count of 0.

B. Find the student who has no score in any of the courses (student name and course number)

A LEFT OUTER JOIN keeps every student, even one with no ENROLLMENT row. The subquery removes students who have a score in at least one course, so only students with no score at all remain.

```
SELECT s.Name, e.CourseNo
FROM   STUDENT s
LEFT OUTER JOIN ENROLLMENT e ON s.RegNo = e.RegNo
WHERE  s.RegNo NOT IN (SELECT RegNo
                       FROM   ENROLLMENT
                       WHERE  Score IS NOT NULL);
```

Output:

Name	CourseNo
Charan	C002
Elango	C003
Gowri	NULL

Result: Charan and Elango are registered for a course but have no score yet. Gowri has no registration at all, so her CourseNo is NULL. Divya is not listed because she has a score (67) in C004, even though her C005 score is empty.

C. Courses taken by faculty 'Kamal' of the 'ECE' department

Two faculty members are named Kamal (F002 in ECE and F005 in CSE), so the department condition is needed.

(i) Inner join

```
SELECT c.CourseNo, c.CourseName, f.FacName, d.DeptName
FROM   COURSE c
INNER JOIN FACULTY    f ON c.FacNo  = f.FacNo
INNER JOIN DEPARTMENT d ON f.DeptNo = d.DeptNo
WHERE  d.DeptName = 'ECE' AND f.FacName = 'Kamal';
```

(ii) Same query as an equi join

```
SELECT c.CourseNo, c.CourseName
FROM   COURSE c, FACULTY f, DEPARTMENT d
WHERE  c.FacNo = f.FacNo
  AND  f.DeptNo = d.DeptNo
  AND  d.DeptName = 'ECE' AND f.FacName = 'Kamal';
```

Output (query i):

CourseNo	CourseName	FacName	DeptName
C002	Digital Electronics	Kamal	ECE
C003	Signals and Systems	Kamal	ECE

Result: Kamal of the ECE department takes the courses C002 (Digital Electronics) and C003 (Signals and Systems). The CSE Kamal's course C004 is correctly excluded.

3. MySQL Program to Retrieve Email-ids (Build an Email List)

Aim

To read the email-id of every employee from the EMPLOYEES table and build a single email list using a stored procedure with a cursor.

Table Creation and Sample Data

```
CREATE TABLE employees (
    emp_id      INT PRIMARY KEY,
    first_name  VARCHAR(20),
    last_name   VARCHAR(20),
    email       VARCHAR(60),
    job_title   VARCHAR(20)
);
```

```
INSERT INTO employees VALUES
(1,'Ram','Kumar','ram.kumar@example.com','Manager'),
(2,'Santhosh','Raj','santhosh.raj@example.com','Supervisor'),
(3,'Hari','Prasad','hari.prasad@example.com','Assistant'),
(4,'Kamal','Nath','kamal.nath@example.com','Engineer'),
(5,'Priya','Devi','priya.devi@example.com','Analyst');
```

Program (Stored Procedure with Cursor)

```
DELIMITER $$
CREATE PROCEDURE build_email_list (INOUT email_list VARCHAR(4000))
BEGIN
    DECLARE v_finished INT DEFAULT 0;
    DECLARE v_email VARCHAR(60) DEFAULT '';

    -- cursor over all email-ids
    DECLARE email_cursor CURSOR FOR
        SELECT email FROM employees;

    -- set flag when no more rows are left
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_finished = 1;

    OPEN email_cursor;
    get_email: LOOP
        FETCH email_cursor INTO v_email;
        IF v_finished = 1 THEN
            LEAVE get_email;
        END IF;
        -- append each email followed by ';'
        SET email_list = CONCAT(email_list, v_email, ';');
    END LOOP get_email;
    CLOSE email_cursor;
END$$
DELIMITER ;

-- Execute the procedure
SET @email_list = '';
CALL build_email_list(@email_list);
SELECT @email_list AS Email_List;
```

Alternative (single query)

```
SELECT GROUP_CONCAT(email SEPARATOR ';') AS Email_List
FROM   employees;
```

Output:

Email_List
ram.kumar@example.com;santhosh.raj@example.com;hari.prasad@example.com;kamal.nath@example.com;priya.devi@example.co

Explanation: The cursor fetches one email at a time. The CONTINUE HANDLER sets v_finished to 1 when the last row has been read, which ends the loop. The INOUT parameter returns the built list to the caller. The GROUP_CONCAT query gives the same list without a cursor (without the trailing semicolon).

Result: The email-ids of all five employees were collected into one semicolon-separated list.

4. ER Diagram and SQL Queries – Student Progress Monitoring System

Aim

To design the ER diagram for a Student Progress Monitoring System and implement it with SQL queries.

Entities, Attributes and Relationships

Entity	Attributes (key underlined in diagram)	Primary key
STUDENT	RegNo, Name, Gender, DOB, MobileNo, City	RegNo
FACULTY	FacNo, FacName, Gender, DOB, DOJ, MobileNo	FacNo
DEPARTMENT	DeptNo, DeptName, DeptHead	DeptNo
COURSE	CourseNo, CourseName, Credits	CourseNo

Relationship	Between	Cardinality	Meaning
Belongs_To	STUDENT – DEPARTMENT	N : 1	A student belongs to one department
Works_In	FACULTY – DEPARTMENT	N : 1	A faculty member works in one department
Offers	DEPARTMENT – COURSE	1 : N	A department offers many courses
Teaches	FACULTY – COURSE	1 : N	A faculty member teaches many courses
Enrolls	STUDENT – COURSE	M : N	Attributes: Semester, Attendance, Marks, Grade (progress data)

ER Diagram

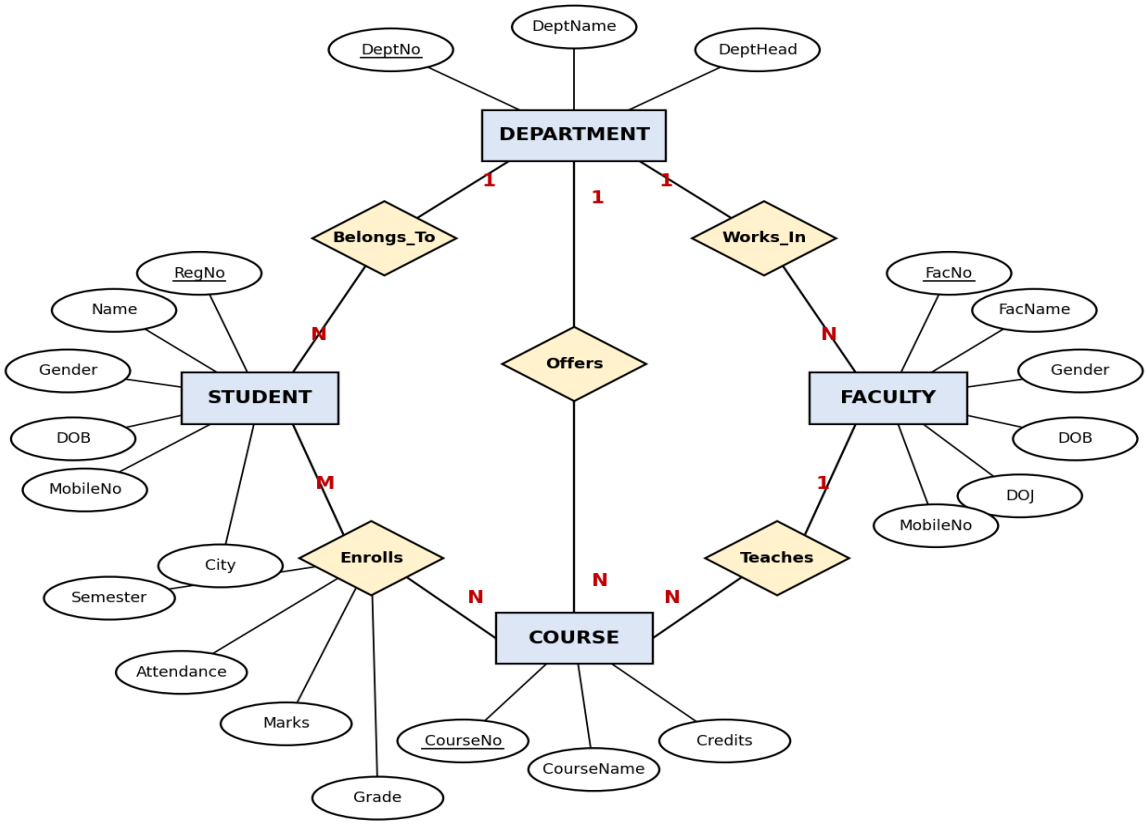


Fig. ER Diagram (Chen notation) – Student Progress Monitoring System

The many-to-many relationship Enrolls becomes a separate table (ENROLLMENT) in the relational schema. It holds the foreign keys RegNo and CourseNo together with the progress data.

Relational Schema (DDL)

```
CREATE DATABASE spms;
USE spms;

CREATE TABLE department (
    DeptNo    VARCHAR(4)  PRIMARY KEY,
    DeptName  VARCHAR(15) NOT NULL UNIQUE,
    DeptHead  VARCHAR(4)
);

CREATE TABLE faculty (
    FacNo     VARCHAR(4)  PRIMARY KEY,
    FacName   VARCHAR(15) NOT NULL,
    Gender    CHAR(1)     CHECK (Gender IN ('M','F')),
    DOB       DATE,
    DOJ       DATE,
    MobileNo  BIGINT,
    DeptNo    VARCHAR(4),
    FOREIGN KEY (DeptNo) REFERENCES department(DeptNo)
);

CREATE TABLE student (
    RegNo     INT         PRIMARY KEY,
    Name      VARCHAR(15) NOT NULL,
    Gender    CHAR(1)     CHECK (Gender IN ('M','F')),
    DOB       DATE,
    MobileNo  BIGINT      NOT NULL,
    City      VARCHAR(15),
```

```
    DeptNo    VARCHAR(4),
    FOREIGN KEY (DeptNo) REFERENCES department(DeptNo)
);

CREATE TABLE course (
    CourseNo    VARCHAR(4)  PRIMARY KEY,
    CourseName  VARCHAR(30) NOT NULL,
    Credits     INT         CHECK (Credits > 0),
    DeptNo      VARCHAR(4),
    FacNo       VARCHAR(4),
    FOREIGN KEY (DeptNo) REFERENCES department(DeptNo),
    FOREIGN KEY (FacNo)  REFERENCES faculty(FacNo)
);

CREATE TABLE enrollment (
    RegNo       INT,
    CourseNo    VARCHAR(4),
    Semester    INT,
    Attendance  DECIMAL(5,2) CHECK (Attendance BETWEEN 0 AND 100),
    Marks       INT          CHECK (Marks BETWEEN 0 AND 100),
    Grade       VARCHAR(2),
    PRIMARY KEY (RegNo, CourseNo),
    FOREIGN KEY (RegNo)    REFERENCES student(RegNo),
    FOREIGN KEY (CourseNo) REFERENCES course(CourseNo)
);
```

Sample Data (DML)

```
INSERT INTO department VALUES
('D001','CSE','F001'), ('D002','ECE','F002'), ('D003','MECH',NULL);

INSERT INTO faculty VALUES
('F001','Ramesh','M','1975-04-12','2005-06-15',9876543210,'D001'),
('F002','Kamal','M','1978-09-23','2008-07-01',9876501234,'D002'),
('F003','Priya','F','1982-01-30','2012-08-20',9845012345,'D001');

INSERT INTO student VALUES
(101,'Arun','M','2004-03-14',9000000001,'Chennai','D001'),
(102,'Bhavana','F','2003-11-02',9000000002,'Madurai','D001'),
(103,'Charan','M','2004-07-21',9000000003,'Trichy','D002'),
(104,'Divya','F','2004-01-09',9000000004,'Coimbatore','D002'),
(105,'Elango','M','2003-12-18',9000000005,'Salem','D003');

INSERT INTO course VALUES
('C001','DBMS',4,'D001','F001'),
('C002','Digital Electronics',3,'D002','F002'),
('C003','Data Structures',4,'D001','F003'),
('C004','Signals and Systems',3,'D002','F002');

INSERT INTO enrollment (RegNo,CourseNo,Semester,Attendance,Marks) VALUES
(101,'C001',3,92,85), (101,'C003',3,88,78),
(102,'C001',3,95,92), (102,'C003',3,70,66),
(103,'C002',3,80,74), (103,'C004',3,72,45),
(104,'C002',3,90,81), (104,'C004',3,85,58);
```

Higher-Level Function – Grade Calculation

A stored function converts marks into a grade (O: 90 and above, A: 80–89, B: 70–79, C: 60–69, D: 50–59, F: below 50). It is then used to fill the Grade column.

```
DELIMITER $$
CREATE FUNCTION GetGrade (p_marks INT) RETURNS VARCHAR(2)
DETERMINISTIC
BEGIN
    DECLARE g VARCHAR(2);
    IF    p_marks >= 90 THEN SET g = 'O';
    ELSEIF p_marks >= 80 THEN SET g = 'A';
    ELSEIF p_marks >= 70 THEN SET g = 'B';
    ELSEIF p_marks >= 60 THEN SET g = 'C';
    ELSEIF p_marks >= 50 THEN SET g = 'D';
    ELSE SET g = 'F';
    END IF;
    RETURN g;
END$$
DELIMITER ;

UPDATE enrollment SET Grade = GetGrade(Marks);
```

Progress Monitoring Queries

Query 1: Average marks of each student

```
SELECT s.RegNo, s.Name, ROUND(AVG(e.Marks),2) AS Avg_Marks
FROM   student s JOIN enrollment e ON s.RegNo = e.RegNo
GROUP BY s.RegNo, s.Name;
```

RegNo	Name	Avg_Marks
101	Arun	81.50
102	Bhavana	79.00
103	Charan	59.50
104	Divya	69.50

Query 2: Students with attendance shortage (below 75%)

```
SELECT s.Name, c.CourseName, e.Attendance
FROM   enrollment e
JOIN   student s ON s.RegNo = e.RegNo
JOIN   course  c ON c.CourseNo = e.CourseNo
WHERE  e.Attendance < 75;
```

Name	CourseName	Attendance
Bhavana	Data Structures	70.00
Charan	Signals and Systems	72.00

Query 3: Topper of each course (sub-query)

```
SELECT c.CourseNo, c.CourseName, s.Name, e.Marks
FROM   enrollment e
JOIN   student s ON s.RegNo = e.RegNo
JOIN   course  c ON c.CourseNo = e.CourseNo
WHERE  e.Marks = (SELECT MAX(Marks) FROM enrollment
                  WHERE CourseNo = e.CourseNo);
```

CourseNo	CourseName	Name	Marks
C001	DBMS	Bhavana	92
C002	Digital Electronics	Divya	81
C003	Data Structures	Arun	78

CourseNo	CourseName	Name	Marks
C004	Signals and Systems	Divya	58

Query 4: Students who failed (grade F)

```
SELECT s.Name, c.CourseName, e.Marks, e.Grade
FROM   enrollment e
JOIN   student s ON s.RegNo = e.RegNo
JOIN   course  c ON c.CourseNo = e.CourseNo
WHERE  e.Grade = 'F';
```

Name	CourseName	Marks	Grade
Charan	Signals and Systems	45	F

Query 5: Department-wise average marks (outer join)

```
SELECT d.DeptName, ROUND(AVG(e.Marks),2) AS Dept_Avg
FROM   department d
LEFT JOIN student    s ON s.DeptNo = d.DeptNo
LEFT JOIN enrollment e ON e.RegNo  = s.RegNo
GROUP BY d.DeptName;
```

DeptName	Dept_Avg
CSE	80.25
ECE	64.50
MECH	NULL

Query 6: View for a student progress report

```
CREATE VIEW student_progress AS
SELECT s.RegNo, s.Name, d.DeptName, c.CourseName,
       e.Attendance, e.Marks, e.Grade
FROM   enrollment e
JOIN   student    s ON s.RegNo    = e.RegNo
JOIN   course     c ON c.CourseNo = e.CourseNo
JOIN   department d ON d.DeptNo   = s.DeptNo;

SELECT Name, CourseName, Marks, Grade
FROM   student_progress WHERE Grade IN ('O','A');
```

Name	CourseName	Marks	Grade
Arun	DBMS	85	A
Bhavana	DBMS	92	O
Divya	Digital Electronics	81	A

Result: The ER diagram was converted into five related tables (DEPARTMENT, FACULTY, STUDENT, COURSE, ENROLLMENT) with primary keys, foreign keys and check constraints. Grades were generated by the GetGrade function, and the queries report averages, attendance shortage, course toppers, failures and department-wise performance.